Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3581784.3607053

RESEARCH-ARTICLE

# FISCO-BCOS: An Enterprise-grade Permissioned Blockchain System with High-performance

**HUIZHONG LI**, University of Chinese Academy of Sciences, Beijing, China

**YUJIE CHEN**, WeBank, Shenzhen, Guangdong, China

**XIANG SHI**, WeBank, Shenzhen, Guangdong, China

**XINGQIANG BAI**, WeBank, Shenzhen, Guangdong, China

**NAN MO**, WeBank, Shenzhen, Guangdong, China

**WENLIN LI**, WeBank, Shenzhen, Guangdong, China

**View all**

**Open Access Support** provided by:

**WeBank**

**University of Chinese Academy of Sciences**

PDF Download
3581784.3607053.pdf
28 February 2026
Total Citations: 42
Total Downloads: 2374

**Published:** 11 November 2023

**Citation in BibTeX format**

SC '23: International Conference for High Performance Computing, Networking, Storage and Analysis
*November 12 - 17, 2023*
*CO, Denver, USA*

**Conference Sponsors:**
SIGHPC


https://doi.org/10.1145/3581784.3607053
ISBN: 9798400701092




# FISCO-BCOS: An Enterprise-grade Permissioned Blockchain System with High-performance

Huizhong Li
lihuizhong21@mails.ucas.ac.cn
ICT/CAS & UCAS
WeBank Blockchain Team
Shenzhen, China

Yujie Chen
yujiechen@webank.com
WeBank Blockchain Team
Shenzhen, China

Xiang Shi
jimmyshi@webank.com
WeBank Blockchain Team
Shenzhen, China

Xingqiang Bai
xingqiangbai@webank.com
WeBank Blockchain Team
Shenzhen, China

Nan Mo
ancelmo@webank.com
WeBank Blockchain Team
Shenzhen, China

Wenlin Li
wenlinli@webank.com
WeBank Blockchain Team
Shenzhen, China

Rui Guo
ruiguo@webank.com
WeBank Blockchain Team
Shenzhen, China

Zhang Wang
octopuswang@webank.com
WeBank Blockchain Team
Shenzhen, China

Yi Sun
sunyi@ict.ac.cn
ICT/CAS & UCAS
Beijing, China

## ABSTRACT

Enterprise-grade permissioned blockchain systems provide a promising infrastructure for data sharing and cooperation between different companies. However, performance bottlenecks seriously hinder the adoption of these systems in many industrial applications that process complex business logic and huge transaction volumes. Our research identifies two key factors that limit the system performance: 1) At the block level, the serial dependency of inter-block processing severely limits the system throughput. A new block must wait for the completion of all previous blocks. 2) At the transaction level, the lack of efficient intra-block transactions concurrency makes it difficult to achieve high performance, especially when dealing with multiple CPU-heavy contracts which are commonly used in industrial scenarios.

In this paper, we present FISCO-BCOS, an enterprise-grade permissioned blockchain system with high performance. To overcome serial limitations and fully utilize machine resources, FISCO-BCOS introduces Block Level Pipelining (BLP) workflow to process blocks in a pipeline manner. In addition, a scheduling algorithm Deterministic Multi-Contract (DMC) is designed to efficiently execute transactions in parallel. Under BLP and DMC, FISCO-BCOS achieves inter-block and intra-block paralleling to meet high-performance requirements in industrial application scenarios. We conducted experiments on two popular test workloads and compared FISCO-BCOS with state-of-the-art platforms in academia and industry such as BIDL and Hyperledger Fabric (HLF). The result shows that FISCO-BCOS achieves 7.4 times and 28.4 times the throughput of BIDL and HLF, respectively, with half the latency of them. BCOS has already been used in over 300 different large-scale industrial scenarios and has become one of the most popular permissioned blockchains.

## KEYWORDS

Enterprise-grade, Permissioned Blockchain, Pipelining Workflow, Deterministic Multi-Contact

**ACM Reference Format:**
Huizhong Li, Yujie Chen, Xiang Shi, Xingqiang Bai, Nan Mo, Wenlin Li, Rui Guo, Zhang Wang, and Yi Sun. 2023. FISCO-BCOS: An Enterprise-grade Permissioned Blockchain System with High-performance. In *The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '23), November 12–17, 2023, Denver, CO, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3581784.3607053

## 1 INTRODUCTION

As a distributed ledger shared by many untrusted participants, blockchain has attracted increasing attention around the world. There are two types of blockchain systems: permissionless (i.e. public blockchain) and permissioned (i.e. consortium blockchain). Bitcoin [38] and Ethereum [22] are two typical permissionless blockchain systems that provide online payment services for individual users. They are built on a decentralized, peer-to-peer network where participants are free to join and leave without trusting each other. Unlike permissionless blockchains, permissioned blockchain systems are designed to enhance mutual trust and improve the efficiency of collaboration between multiple parties in industrial scenarios [53]. Several permissioned blockchain platforms have been built to support the growing number of such requirements, e.g., HyperLedger Fabric (HLF) [3], Quorum [14].

In permissioned blockchains, participants need to be authenticated to join the network. Generally, permissioned blockchains have a higher performance than permissionless ones because they have a limited number of participants and run more efficient byzantine fault tolerant (BFT) algorithms [12, 13, 54, 62] as consensus



This work is licensed under a Creative Commons Attribution International 4.0 License.

*SC '23, November 12–17, 2023, Denver, CO, USA*
© 2023 Association for Computing Machinery.
ACM ISBN 979-8-4007-0109-2/23/11…$15.00
https://doi.org/10.1145/3581784.3607053

protocols. With the help of the permissioned blockchain, organizations with common goals can revamp transparency, accountability, and workflow together to significantly save temporal and monetary costs of mutual cooperation [9]. In recent years, permissioned blockchains have been widely used in finance, trade, logistics, and other industrial scenarios[17, 30, 33, 52].

Through the experiences of applying permissioned blockchains to industrial scenarios, we discover that high performance is the basic requirement for permissioned blockchains. For instance, some financial systems are desired to handle more than 65,000 transactions per second (tps) [55] along with sub-second latency [40]. However, the performance bottlenecks significantly hinder the leverage of current permissioned blockchains in these industrial scenarios. HLF [3] and Quorum [14] are the two most popular enterprise-grade permissioned blockchain systems worldwide, while the throughput and latency of these systems are about 3,500 tps, 600 ms [3] and 2,100 tps, 2s [8] for SmallBank workload [19] respectively. In recent years, researchers in academia have proposed some solutions[28, 44, 46–48, 61] to improve the performance of permissioned blockchains. For example, FastFabric [28] re-architected HLF and increased its throughput to 20,000 tps, and BIDL [46] achieved 41,000 tps in a datacenter network through parallel execution and consensus. These works make a progress in permissioned blockchain performance optimization. But many real industrial applications in payments, smart city governance, etc. require much higher performance (e.g. Visa's 65,000 tps [55]), the current solutions still struggle to meet these scenarios.

By conducting a large number of experiments and taking in-depth measurements, we observe two key issues that seriously affect the performance of existing permissioned blockchain systems.

- **For inter-block handling**, the serial dependency of inter-block processing severely limits the system throughput. The processing of a block usually goes through multiple phases, such as $create$, $sequence$, $execute$, $validate$, and $commit$. The order of these phases varies from system to system. However, in traditional permissioned blockchains, such as Quorum [14], Diem [5], etc, the creation of a new block must wait for all the previous blocks to complete their entire flow. In this way, the system can only process one block at a time, resulting in deficient performance.
- **For intra-block processing**, transaction execution is the most time-consuming task but effective concurrency mechanisms are lacking in current systems. In real industrial scenarios, there are much more complex contracts that require more resources in the execution phase. Early HLF [3] tries to solve this problem by proposing a parallel mechanism to execute transactions in the endorsing phase, however, it leads to a high rate of transactions aborts in the validation phase when transactions access conflicting resources. Some recent approaches [1, 7, 18, 64] introduce static analysis and speculative execution of contracts to process intra-block transactions concurrently. Unfortunately, it is very difficult to deterministically infer all dependencies across smart contracts. Moreover, the waiting for dependencies construction significantly constrains the parallelism of the overall system as well.

To address the above challenges, we present FISCO-BCOS, an enterprise-grade permissioned blockchain system with high performance. FISCO-BCOS designs a Block Level Pipelining (BLP) workflow to break the serial dependency between blocks and processes them in a pipeline with four stages. Some blocks are processed at one stage, while other blocks are processed simultaneously at different stages. In addition, Deteministic Multi-Contract (DMC) is introduced to execute transactions concurrently within a block. Transactions are dispatched into multiple shards and processed in parallel by a group of executors. As a result, FISCO-BCOS achieves inter-block and intra-block parallelism so that can provide high performance in enterprise scenarios. We implemented FISCO-BCOS and evaluated its performance compared to BIDL [46], the state-of-the-art work in academia, and HLF [3], the most commonly used system in the industry. The result shows that FISCO-BCOS achieves 7.4 times and 28.4 times the throughput of BIDL and HLF, respectively, with half the latency of them. FISCO-BCOS has been widely used in more than 300 real-world business applications and has become one of the most popular permissioned blockchains. Taking the *Mutual Health Code Recognition System* as an example, built on top of FISCO-BCOS, it has supported over 300 million cross-border travels during its service period. More application cases will be shown in Sec. 6.

In summary, the main contributions of this paper are as follows:

(1) We propose an inter-block processing approach, Block Level Pipelining (BLP), that breaks the serial dependency of block processing and handles blocks in a pipelined manner throughout their lifetime.
(2) We present an intra-block scheduling algorithm, Deterministic Multi-Contract (DMC), which dispatches transactions into several shards and leverages a set of executors to process transactions in each shard concurrently.
(3) We have developed a full-fledged high-performance permissioned blockchain system, FISCO-BCOS, and made it available at Github[1]. As an open-source enterprise-grade permissioned blockchain systems, FISCO-BCOS is widely used in various industrial scenarios.

## 2 BACKGROUND

We briefly introduce the current permissioned blockchain workflow and the techniques of smart contract concurrency in this section.

### 2.1 Permissioned Blockchain Workflow.

Same as permissionless blockchains such as Bitcoin [38] and Ethereum [22], permissioned blockchains receive transactions from clients, go through a workflow to update the ledger, and ensure consistency among replicas under a trustless environment. According to their workflows, permissioned blockchains can be generally divided into two different categories. The first is the $Execute \rightarrow Order \rightarrow Validate$ (EOV) workflow. Typical systems with this workflow are HLF [3] and its optimizations [28, 47, 48]. In this workflow, execution nodes (endorsers) execute the transactions concurrently by the optimistic concurrency control (OCC) mechanism, then ordering nodes (orderers) make a consensus on the

[1] https://github.com/FISCO-BCOS/FISCO-BCOS.

sequence of the transactions, batch them into a block, and finally return the block to execution nodes for verification and commit. One of the disadvantages of this workflow is the high rate of transaction aborts when transactions access resources in conflict, thus hard to achieve high performance in real-world applications. The second is the $Order \rightarrow Execute$ (OE) workflow. Systems like Quorum [14] leverage such workflow to process transactions. Under this mechanism, nodes first make an agreement on the orders of transactions. After that, each node executes transactions based on the established sequences. As a result, transactions tend not to be aborted because of contention. Unfortunately, due to the serial process of blocks, these systems generally do not provide sufficient performance for many real-world applications.

## 2.2 Smart Contract Concurrency.

There are many smart contracts programming languages in permissioned blockchain systems, e.g., Solidity [23], Move [35], and ink! [43]. Among them, Solidity, introduced by Ethereum [22] platform, run in the Ethereum Virtual Machine (EVM) [21], is the most wildly used one. Smart contracts programmed with Solidity can be thought of as a collection of self-defined states and functions that manipulate the states. Each contract has a separate storage space and communicates with other contracts through function calls. Once the contracts are deployed in the blockchain, we can access them by sending transactions that invoke the public functions of these contracts, through which the ledger is updated. The execution of a transaction involves invocations of one or more contracts and is generally the most time-consuming phase of the entire block-processing workflow.

Inspired by the design of database systems, many studies [1, 4, 7, 10, 18, 31, 64] have been proposed to add concurrency to smart contracts. In these approaches, transactions within a block are handled as follows: i) the miner generates a transaction dependency graph (usually a directed acyclic graph) by static analysis of contracts or speculative execution of transactions; ii) the miner sends transactions and the graph to validators; iii) validators execute transactions in parallel according to this graph. These efforts have greatly improved the efficiency of transaction execution, but two limitations remain. On the miner side, static analysis of contracts is challenging when dynamic cross-contract invocations occur since the contract access patterns are not predictable. Moreover, speculative execution may lead to high rollback and low parallelism when multiple complex contracts call each other. From the perspective of the validator, the performance is limited by waiting to receive the block and dependency graph.

To sum up, we find that the inter-block processing workflow and intra-block transactions concurrency used by current permissioned blockchain systems face various performance limitations. To deliver an enterprise-grade permissioned blockchain, we must design a more efficient workflow and concurrency mechanism to achieve high performance.

# 3 SYSTEM OVERVIEW

FISCO-BCOS is designed to be an enterprise-grade permissioned blockchain with high performance to support industrial scenarios. In this section, we provide an overview of our system.

**Threat Model.** FISCO-BCOS is composed of a group of participants with two different roles: *node* and *client*. All participants are identified and managed by explicit secret/public key pair, which is generally used by permissioned blockchains[3, 14, 46]. Any two nodes $\langle N_i, N_j \rangle$ are connected by point-to-point bidirectional communication channels, constructing a full-meshed P2P network. FISCO-BCOS adopts the BFT [13, 54] protocol to achieve consensus, thus the network contains $3f + 1$ nodes, where at most $f$ nodes may be malicious. A client $C_j$ signs a transaction $T$ with its secret key $\sigma_j$ and submits it to the network. Upon receiving the signed transaction, node $N_i$ checks the signature to ensure the transaction is valid then pushes it into the transaction pool and propagates to the other nodes. Afterward, transactions are processed through a workflow shown in Figure 1.

**Workflow.** To prevent transactions from being aborted due to conflicting access to resources, FISCO-BCOS puts transactions ordering prior to execution. In addition, FISCO-BCOS divides the validation phase into two phases, checkpoint and commit, where the checkpoint phase is network-intensive and the commit phase is io-intensive. In this way, both network and IO resources are fully utilized. Therefore, the workflow of FISCO-BCOS consists of four stages that cover the entire lifetime of one block. Here we describe the four stages of this workflow, and we will give its optimization later.

*(1) Ordering.* This stage is mainly responsible for transaction ordering. Nodes batch transactions into a *generated* block $B_h^g$ ($h$ denotes block number), then run a BFT consensus protocol to agree on the sequence of transactions and generate a *proposed* block $B_h^p$. The consensus protocol is pluggable so that different varieties of BFT algorithms (e.g., PBFT[13], Hotstuff[62]) adopted in permissioned blockchains can be all applied in FISCO-BCOS as well.

*(2) Execution.* This stage executes transactions of the *proposed* block and yields a new state of the ledger. For instance, the execution of $B_h^p$ creates new state $S_h$ from the previous state $S_{h-1}$ (Figure 1).

*(3) Checkpoint.* FISCO-BCOS leverages a lightweight protocol to verify that the execution results are consistent across nodes. Concretely, each node broadcasts its execution result $B_h^e$ and prepares an *approved* block $B_h^a$ to commit after collecting enough unanimous result. However, if a node does not collect enough unanimous $B_h^e$ before timeout, the exception handling mechanism would be triggered. With this mechanism, FISCO-BCOS drops the illegal transaction and returns to stage 2 for re-execution. Sec. 4.3 illustrates the mechanism in detail.

*(4) Commit.* Upon receiving the *approved* block $B_h^a$, FISCO-BCOS starts a process to write the block, after which the *committed* block $B_h^c$ can be retrieved from clients.

**Optimization.** Based on the above workflow, each stage performs tasks on blocks that are in different lifetimes, and thus different stages rely on different resources. In particular, the ordering and checkpoint stage heavily depend on network resources due to the communication between nodes, while the execution stage relies strongly on CPU resources for computation; and the commit stage is disk I/O intensive respectively. To improve the performance of block processing, we propose two significant mechanisms.

Firstly, we design an inter-block processing mechanism, Block Level Pipelining (BLP), that processes multiple blocks in a pipelined

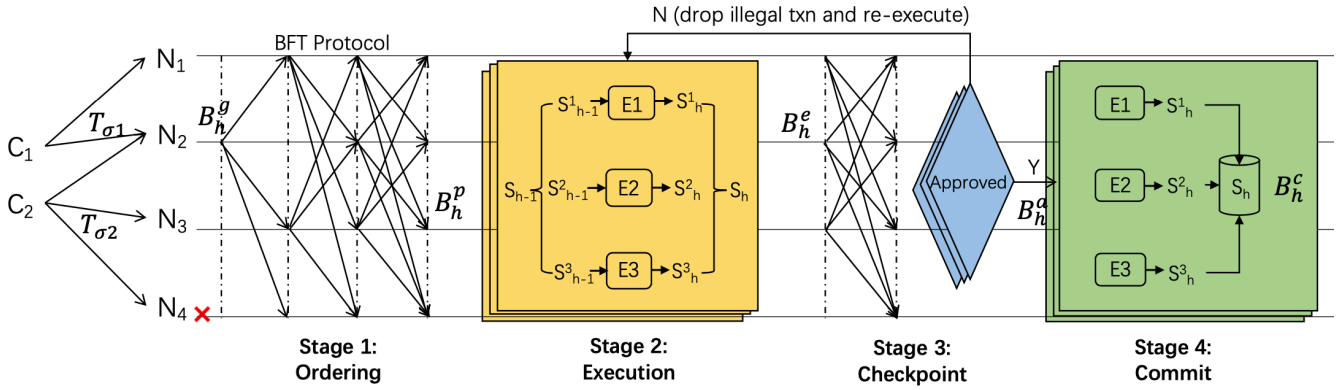



**Figure 1: The overall workflow of FISCO-BCOS.**

manner throughout their lifetime. With this approach, some blocks are processed at one stage, while others are processed at different stages. Secondly, since the execution stage is generally the most time-consuming in the workflow, to improve intra-block handling efficiency, FISCO-BCOS designs a scheduling algorithm, Deterministic Multi-Contract (DMC), to execute transactions in parallel. DMC splits intra-block transactions into several shards and dispatches them to a group of executors for parallel processing. In addition, a mechanism to detect and resolve conflicts is designed for contract calls across shards.

In this way, FISCO-BCOS provides not only inter-block improvements that break the serial dependency of block processing but also intra-block enhancements that efficiently handle complex transactions, greatly improving the overall system's performance. We will illustrate the details of these mechanisms in the next two sections.

## 4 BLOCK LEVEL PIPELINING

According to the previous introduction, in order to make full use of network/cpu/io resources and enhance the entire system throughput, FISCO-BCOS introduces Block Level Pipelining(BLP) to optimize the workflow and process several blocks simultaneously. Furthermore, when applying BLP workflow to FISCO-BCOS, three key problems need to be addressed: (1) How to control the pipeline to balance the use of various resources? (2) How to maintain in-memory states so that FISCO-BCOS can execute blocks based on previously uncommitted blocks? (3) How to ensure that all nodes submit the same result and no fork[2] occurs. We describe them in the following sub-sections.

### 4.1 Pipeline Control

In our workflow, two consecutive stages perform a producer-consumer pair, where the former stage produces blocks carrying new context and transforms them into the next stage. Balancing block production and consumption between stages is crucial so that the pipeline can work efficiently. Define $\zeta_i$ as the number of blocks in stage $i$, then the length of the pipeline is $\mu = \sum_{i=1}^{4} \zeta_i$. The ideal pipeline status is always $\zeta_i = 1$ for all $i$ (i.e. one block per stage). However, due to the imbalance of network, computation, and storage capacity, it is difficult to achieve the ideal state in a practical environment. Instead of struggling to reach the ideal status, we introduce a sliding window algorithm to control the pipeline dynamically. The algorithm adjusts the block-generating speed to balance the ordering and execution stages, by which makes full use of network and computation resources. Specifically, according to the number of *proposed* blocks in stage 2 (i.e. $\zeta_2$, which are being or waiting to be executed), the algorithm adjusts the number of *generated* blocks in stage 1 (i.e. $\zeta_1$). Let $\zeta_1$ denote the sliding window size, and $\lambda$ denotes the threshold of $\zeta_1$, the algorithm runs three situations as shown in Algorithm 1. In the first situation, where $\zeta_2$ equals 0, it means that the execution is faster than the ordering, resulting in an idle execution phase, so we need to propose more blocks by doubling $\zeta_1$. In the second situation, where $\zeta_2$ is less than $\lambda$ (but greater than 0), we reduce the proposing growth rate to balance ordering and execution. In the last situation, where $\zeta_2$ is greater than $\lambda$, it means that the execution phase is overloaded, so we suspend proposing, then reset $\zeta_1$ to 0.

**Algorithm 1:** Sliding Window Algorithm

```
Function adjustPipeline():
    if ζ2 = 0 then
        ζ1 = min (2 * ζ1, λ);
    else if ζ2 < λ then
        ζ1 = ζ1 + 1;
    else
        ζ1 = 0;
End Function
```

In addition, to balance the execution and the last two stages, FISCO-BCOS supervises the number of *executed* and *approved* blocks with $\mu' = \sum_{i=3}^{4} \zeta_i$ to control the production capability of the execution stage. Block execution is blocked when $\mu'$ exceeds the threshold $\lambda'$. And $\lambda'$ is related to the memory consumption so that it can be configured according to the machine's memory.

[2]There are two blocks with the same number but different results are committed by two nodes

## 4.2 In-memory States

This section discusses the cache strategy used by FISCO-BCOS to manage in-memory states during block execution. Under BLP workflow, a block's execution is based on both previously committed and uncommitted blocks' states, therefore FISCO-BCOS employs a two-layer cache system as shown in Figure 2. The first layer (L1) is dedicated to uncommitted blocks (which can be rolled back) and consists of linked in-memory blocks, each storing the states modified by its respective block. The second layer (L2) is for committed blocks (which are persistent) and uses a traditional LRU-based cache. This approach enables efficient management of in-memory states during block execution in FISCO-BCOS.

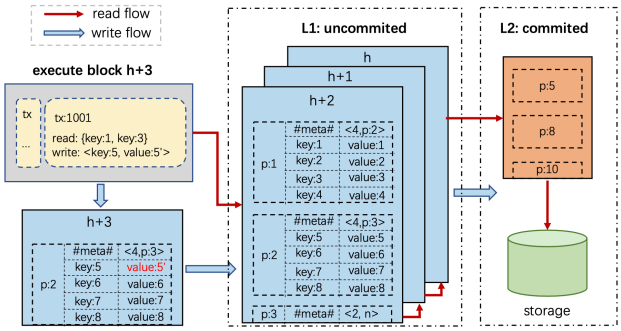



**Figure 2: Two-layer cache design.**

As the read flow in Figure 2, when a block wants to read a key, it first searches the L1 cache for that key. If the corresponding key-value pair is not found in L1, FISCO-BCOS searches the L2 cache, followed by persistent storage, until it finds the requested key-value pair. Regarding the writing process (write flow in Figure 2), FISCO-BCOS stores the updated keys of a block in a separate new cache during execution. After completing the execution of the entire block, these updated keys are moved to the L1 cache, made read-only. After the checkpoint phase is approved, the block is committed to L2 cache and storage (or rolled back if it is not approved). Note that we only store the states that a block writes and ignore the ones it reads (copy-on-write mechanism [51]) in L1 for memory optimization. Furthermore, we design *KeyPage* for contract data storage to reduce disk I/O operations. A contract's data is managed by several contiguous pages, each of which stores a set of key-value.

## 4.3 Checkpoint Protocol

To ensure that all nodes submit the same result after executing a block without forking, FISCO-BCOS uses a checkpointing protocol to achieve consistency in execution results across nodes. Unlike the traditional BFT[13, 54] and CFT[42] protocols, which require three and two rounds of broadcasts, respectively, this protocol requires only one round of communication and is therefore very lightweight and efficient. Meanwhile, the protocol still maintains safety and liveness.

There is no single primary node in this protocol. Each node broadcasts its *executed* block $B_h^e$ to other nodes, and reaches consistency after receiving $\theta$ unanimous results before timeout. For safety, we set $\theta = \lceil 1.5f + 1 \rceil$ (i.e. half of the total nodes $3f + 1$), thus it is impossible for two blocks with the same number but different results to be approved simultaneously.

In terms of liveness, FISCO-BCOS introduces exception handling when the nodes cannot reach a consensus before the timeout. This occurs when there are non-deterministic transactions in the block that result in diverse state updates. Note that non-deterministic transactions rarely occur and can only be forged by malicious nodes or due to static analysis errors in smart contracts. To address this situation, each node in FISCO-BCOS records the execution results for each transaction in the current block. Meanwhile, we leverage the Algorithm 2 to find the non-deterministic transaction, drop it, and re-execute the block.

**Algorithm 2:** Checkpoint Exception Handling

**Input** : *Executed block* that fails to be validated: $B_h^e$ ,
All the consensus nodes: *nodes*
**Output**: New *executed block*: $B_h^e$

```
1  Function handleException(B_h^e, nodes):
2      txs = B_h^e.getTxs();
3      f = getMaxFaultyNodes();
4      foreach tx in txs do
5          count = 0;
6          foreach node in nodes do
7              tx_wset = node.getTxWSetHash (tx.id());
8              if tx_wset != tx.wset() then
9                  count++;
10                 if count > f then
11                     B_h^p = dropIllegalTx (B_h^p, tx);
12                     B_h^e = reExcute (B_h^p);
13                     return B_h^e;
14 End Function
```

As the algorithm shows, FISCO-BCOS leverages the invalid block as well as all the consensus nodes to handle the exception. First, FISCO-BCOS obtains all the transactions from the invalid *executed* block $B_h^e$ (Line 2). For each transaction, FISCO-BCOS calculates the number of nodes that give out different results from the one recorded in the block (Lines 5-9). Then, it checks if there are more than $f$ nodes (which means there is at least one honest node) with different transaction results (Line 10). If yes, the transaction is illegal, leading to consensus failure in the checkpoint phase. FISCO-BCOS then drops the illegal transaction from *proposed* block $B_h^p$ and re-executes the block (Lines 11-12). Finally, the algorithm returns the new block result $B_h^e$ (Line 13).

# 5 DETERMINISTIC MULTI-CONTRACT

As previously mentioned, FISCO-BCOS utilizes the BLP workflow to achieve inter-block parallelism and process multiple blocks effectively. However, the execution stage can be time-consuming, particularly in real-world business scenarios where many complex contracts are deployed in a blockchain system. This can result in execution becoming a bottleneck in the workflow. Therefore, an efficient intra-block transaction processing mechanism should be introduced to further improve the performance of FISCO-BCOS.

In FISCO-BCOS's workflow, we put the ordering stage ahead of the execution, avoiding transaction aborts. Apart of this, in the execution stage, all nodes can start executing intra-block transactions simultaneously since the block has been agreed in the previous stage, eliminating the waiting between the miner and validators occurring in prior approaches [4, 10, 18, 64]. Furthermore, we design a parallel mechanism named Deterministic Multi-Contract (DMC) which dispatches transactions of intra-block into several shards and enables multiple executors to process transactions in parallel. As an example shown in Figure 3, during the DMC execution phase, transactions that invoke different contracts (e.g., $T_1$ and $T_4$ call $/app1/c1$ and $/app2/c3$ respectively) are first dispatched into three shards, each of which is assigned to a specific executor. Three executors concurrently process transactions in their respective isolated contexts.

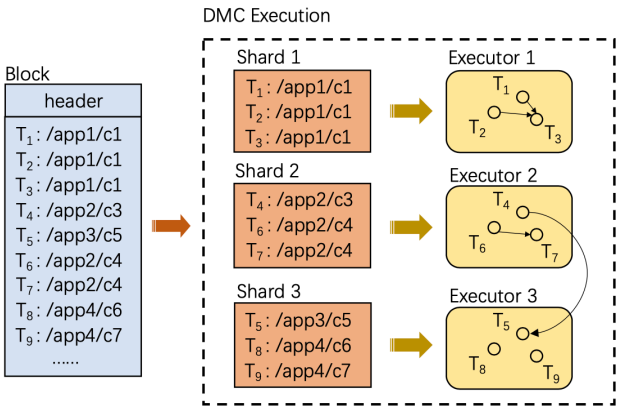



**Figure 3: An example of one block being processed by DMC.**

With the DMC mechanism, FISCO-BCOS achieves a high degree of parallelism in execution, therefore greatly improving the efficiency of the intra-block process. However, since all nodes process their intra-block transactions with DMC independently and must produce a consistent result, in addition to ensuring the efficiency of DMC, it must also ensure that it is deterministic. To this end, three problems need to be solved: (1) designing an efficient and deterministic scheduling algorithm for parallel execution of multiple transactions, (2) handling the case where parallel transactions have conflicting resource accesses, and (3) guaranteeing atomicity of commits among all executors. In the following subsections, we will show our solutions in detail.

## 5.1 Parallel Scheduling

The execution of a transaction involves invocations of one or more contracts. Each contract has a separate storage space and communicates with other contracts through function *calls*. Therefore we can observe that there are two types of transactions in FISCO-BCOS, one that only invokes contracts within a shard and the other that invokes contracts across shards. To achieve deterministic results, we use frequent global barriers to sequence the *calls* between shards. In particular, DMC schedules transactions by the contract address they invoke, and runs for several rounds until all transactions have been fully executed. In each round, all transactions with different contract addresses are assigned to a group of shards and processed by their respective executors. Through several rounds, all transactions in the block are completely processed, and the execution ends. To design an efficient and deterministic scheduling mechanism, there are two concerns we need to address: (1) how to design an algorithm to parallelize transactions that call contracts within a shard only, and (2) how to design a scheduling method to parallelize transactions invoking contracts across shards.

For convenient expression, we use a pair $\langle T_n, c1\rangle$ to represent that transaction $T_n$ invokes contract $c1$. While $\langle T_n, c1 \rightarrow c2\rangle$ means the transaction $T_n$ relies on the contract $c1$ and $c1$ calls another contract $c2$.

**Parallel scheduling within a single shard.** In each round, the executor analyzes transactions assigned to this shard and obtains resources they access, based on which, it constructs a directed acyclic graph (DAG) according to the resources dependencies among transactions. Prior works [26, 29, 45] have provided many effective strategies for transaction dependencies analysis. With the DAG, the executor executes these transactions without resource conflicts in a deterministic parallel manner, significantly improving the intra-block execution. As shown in Figure 4, three transactions $\langle T_1, /app1/c1\rangle$, $\langle T_2, /app1/c1\rangle$, and $\langle T_3, /app1/c1\rangle$ are concurrently executed following the DAG guidance in $E1$.

**Parallel scheduling across shards.** For these transactions, each executor processes transactions independently until a cross-shard call occurs. At this time, the execution of this contract (caller) is interrupted and waits until the execution of another contract in called shard (callee) finishes. As shown in Figure 4, $\langle T_5, /app3/c5 \rightarrow /app2/c3\rangle$ is interrupted in round 1, due to contract $/app3/c5$ makes a *call* of contract $/app2/c3$, and therefore DMC launches an interrupted transaction to be executed in round 2. After the external call finishes, the executor $E2$ will recover the context restored, changing the contract address back to $/app3/c5$, and continue executing contract $/app3/c5$ in round 3. Note that we use a similar $wait-resume$ mechanism like interruption processing in the operating system, providing a high degree of parallelism in contract calls across shards.

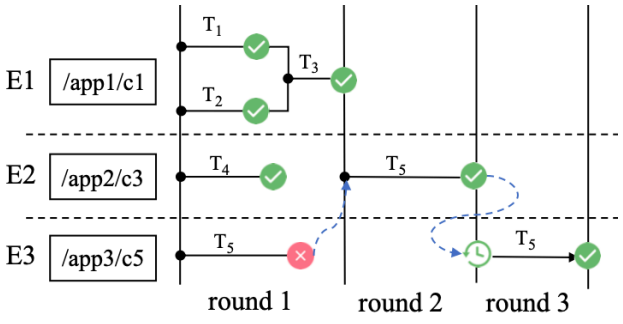



**Figure 4: The scheduling process of 5 transactions that invoke 3 various contracts. Three transfer transactions $T_1$, $T_2$, and $T_3$ are concurrently executed following the DAG guidance in $E1$. $T_4$ and $T_5$ are assigned to $E2$, $E3$ respectively, and $T_5$ invokes a cross-contract call of $/app2/c3$.**

## 5.2 Conflict Resolution

The second issue that the DMC mechanism may face is conflict resolution. There are two types of conflicts in FISCO-BCOS transaction processing: *(1) access conflict; (2) deadlock.* The former happens when two transactions, $\langle T_i, /app1/c1 \rightarrow /app3/c3\rangle$ and $\langle T_j, /app2/c2 \rightarrow /app3/c3\rangle$, are executed in parallel by two different executors. In the first round, each executor executes contracts $/app1/c1$ and $/app2/c2$ in its shard, respectively. However, in the following round, they both try to access contact $/app3/c3$ across shards which leads to conflict. As for the latter case, it happens when a transaction $\langle T_i, /app1/c1 \rightarrow /app2/c2\rangle$ and another one $\langle T_j, /app2/c2 \rightarrow /app1/c1\rangle$ are processed by two executors in parallel. In this case, two executors wait for each other to finish their task first in round 2, resulting in a deadlock.

To detect and resolve these conflict situations, FISCO-BCOS adds a lock on every key of states that is read or written by a transaction. In that way, when there is an access conflict, the executor can easily find that the key needed by the transaction $\langle T_j, /app2/c2 \rightarrow /app3/c3\rangle$ has already been locked by the transaction $\langle T_i, /app1/c1 \rightarrow /app3/c3\rangle$). In common pessimistic concurrency control (PCC) mechanisms, either of two transactions can release their locks to resolve access conflicts. However, this approach cannot be used in our system because we must provide a deterministic conflict resolution. We use a global barrier to sort these transactions when they invoke a contract across shards. Conflicting transactions (accessing the same contract) are queued by the transaction index in the block (if $i < j$, $T_i$ comes before $T_j$) and wait to be scheduled in order.

In terms of the deadlock situation, each transaction context uses two lists to record the locking situation of state keys. In particular, a *locked_key_list* records the keys that are locked by the current context. And a *wait_key_lock_list* stores the locked keys that are needed by the current context. For each round of scheduling, the scheduler constructs a dependency graph from *locked_key_list* and *wait_key_lock_list*, by which it can infer a deadlock if there is a circle in the graph. FISCO-BCOS caches the transaction execution result of each round with $< t, r, ctx >$, where $t$, $r$ and $ctx$ denote transaction, round, and context, respectively. To resolve the deadlock, FISCO-BCOS reverts the transaction that has a larger index to the context of the last round and pushes it into a *blocking_tx_list*. After all other transactions are executed, FISCO-BCOS pops transactions from *blocking_tx_list* to resume the execution.

## 5.3 Commit Atomicity

The last issue is to guarantee atomicity during the commit stage. Because DMC utilizes multiple executors with independent contexts to process intra-block transactions, it must ensure that a block's execution results from these executors are either all committed or not committed. To address this problem, FISCO-BCOS proposed a two-phase commit algorithm.

Algorithm 3 shows the process of the commit. The algorithm takes in the DMC scheduler, all the executors, and the current block header. First, the scheduler gets a primary key from the storage (Line 2). After that, the scheduler pre-writes the block header guided by the primary key (Line 3). The scheduler then broadcasts a 'NOTIFY' message to all executors (Line 4). After receiving the message, executors start writing its execution result to the storage (Lines 5-11). If some exceptions occur in this process, the scheduler will rollback the commit and try to re-commit later. If an executor successfully stores its results, it will then notify the scheduler that it has accomplished its work. Finally, the scheduler will commit to the storage after receiving the notifications from all executors (Line 12). In this way, the DMC mechanism ensures the commit atomicity. Meanwhile, this algorithm also reduces the overhead of the commit stage since multiple executors commit in parallel (Lines 5-9).

**Algorithm 3:** Two-phase Commit

```
   Input  : The DMC scheduler: scheduler,
            All the executors: executors,
            Current block header: H_h
1  Function commit(scheduler, executors, H_h):
2      pKey = scheduler.getPrimaryKey ();
3      scheduler.preWrite (pKey, H_h);
4      scheduler.broadcast (executors,'NOTIFY');
5      foreach e in executors do
6          if !e.preWrite (pKey,e.getResult ()) then
7              scheduler.rollback ();
8              return;
9          e.notify (scheduler);
10     scheduler.commitToStorage ();
11 End Function
```

# 6 IMPLEMENTATION AND APPLICATIONS

We have implemented FISCO-BCOS in C++ (200,000+ lines of code) using a plug-in framework where most core modules can be replaced on demand, including cryptography algorithms, consensus protocols, contract virtual machines, etc. By default, we use libsecp256k1 [11] for ECDSA signatures and secret/public key operations. We upgrade the PBFT [13] protocol to support transaction ordering and block generation. In addition to the traditional EVM [21], the WASM [57] engine is integrated for transaction execution as well. We utilize RocksDB [36] to store archived blocks, transactions, and states. Besides, the distributed storage engine TiKV [6] is also introduced to support storage scaling.

Accoding to the stastistic from recent blockchain application whitepaper, 300+ real-world business blockchain applications have been successfully deployed to serve for 4000+ enterprises based on FISCO-BCOS. These cases cover more than 16 important business scenarios, such as cross-border collaboration, judicial services, financial services, smart governance, supply chain management, etc [24, 25]. We show some typical cases here.

**Cross-border collaboration.** The *Mutual Health Code Recognition System*, implemented based on FISCO-BCOS, is deployed both in Macau and Guangdong to support travelers crossing between two areas. The platform has supported over 300 million travelers crossings during its service period (30 months) [24].

**Judicial Services.** WeBank, YIBI Technology and Arbitrators have partnered to launch a blockchain-based evidence and arbitration platform to provide real-time, high-performance, authentic,

and traceable data for judicial services. With the help of FISCO-BCOS, the platform standardizes evidence and trial processes to achieve the requirements of authenticity, legality, and relevance of evidence. The platform has processed 3 billion pieces of co-deposited evidence [24].

**Financial Services.** With the help of FISCO-BCOS, WeBank, together with its partner banks, has established a permission blockchain-based inter-institutional reconciliation platform (shown in Figure 5). The platform stores business information, including funds and transactions, in the form of copies on the chain for reconciliation purposes. Since its launch, the platform has accumulated over 200 million transactions [24].

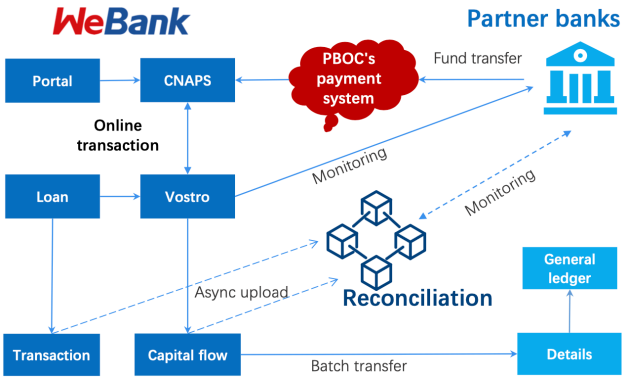



**Figure 5: Inter-bank reconciliation platform.**

**Smart Governance.** With the distributed, transparent and hard-to-tamper characteristics of blockchain, based on the FISCO-BCOS platform, the Identification Bureau, Telecom, and Polytechnic Institute of Macau build a permission blockchain platform to implement the electronic flow of cross-departmental information of the government and double the efficiency of government services [24].

**Supply Chain Management.** Based on the FISCO-BCOS platform, GRGBanking has built a supply chain finance platform that realizes a multi-level split and flow model of accounts receivable. The platform transforms accounts payable of core enterprises into electronic vouchers for payment and financing through smart contracts, realizing multi-level transmission of core enterprises' credit and benefiting multiple parties in the industrial chain. The platform has access to 6 banks and one factoring institution, and the capital flow in the industrial chain exceeds 1.5 million dollars [24].

# 7 EVALUATION

We design FISCO-BCOS as an enterprise-grade permissioned blockchain system, so it is important to evaluate the performance of FISCO-BCOS in different enterprise environments. In addition, FISCO-BCOS introduces inter-block pipelining and intra-block parallelization, and it is essential to analyze the effectiveness of these mechanisms. Therefore, we conduct a series of experiments on FISCO-BCOS, seeking to answer the following research questions:

- How does FISCO-BCOS perform in different environments?
- How do BLP and DMC work in FISCO-BCOS respectively?
- Is FISCO-BCOS scalable in real-world scenarios?

## 7.1 Experiment Setup

We first introduce the basic information about our experiment setup.

**Baseline.** We compare FISCO-BCOS with two permissioned blockchain platforms, HLF[3] and BIDL [46]. HLF is the most popular enterprise-grade permissioned blockchain platform that is world-widely used in various scenarios [30]. BIDL is a state-of-art work in academia that uses a shepherded parallel workflow to achieve high performance in datacenter networks.

**TestBed.** We deploy two typical scenarios for testing FISCO-BCOS and Baseline solutions in Cloud environments. First, a small-scale testbed consists of 11 virtual machines (1 as client and 10 as nodes), each of which is equipped with an Intel(R) Xeon(R) Platinum 8378C CPU @ 2.80GHz (16 cores, 32 threads), 64 GB of RAM, and Ubuntu 20.04. The communications of IP multicasting [60] between VMs are equipped to support BIDL [46]. In addition, we also deploy a large-scale testbed that contains 100 less-configured nodes with 8vCPUs (4 cores, 8 threads).

**Workloads.** We conduct evaluations by using the popular blockchain benchmark BlockBench [19], which contains both macro-benchmark workloads for evaluating the overall performance and micro-benchmark workloads for evaluating the performance of individual layers. In particular, we use SmallBank(macro-benchmark workload) which creates a group of accounts and performs random transfers among them to evaluate performance for IO-intensive scenarios, and CPUHeavy (micro-benchmark workload) which performs a quick sort over an array of integers to simulate the performance for computation-intensive scenarios.

## 7.2 End-to-end Performance Comparison.

We evaluate the end-to-end performance of FISCO-BCOS, HLF, and BIDL in the small-scale testbed.

**Performance in conventional environments.** Firstly, we perform an end-to-end performance comparison over the SmallBank workload. In this experiment, we randomly generate a batch of *transfer* transactions, each involving two accounts belonging to different organizations. As shown in Figure 6, FISCO-BCOS outperforms HLF and BIDL in both throughput and latency. For throughput, HLF processes transactions in a parallel fashion in the endorsing phase, however, in the ordering and validation phase, it must process transactions and blocks serially so that limit its throughput. BIDL on the other hand introduces a parallel workflow to enhance the efficiency of transactions consensus and execution and therefore achieves high throughput than HLF. Our FISCO-BCOS achieves the highest throughput with the BLP and DMC mechanisms, i.e. 93.82K Txn/s, more than 7.4 times over BIDL[3] (12.6K Txn/s) and more than 28.4 times over HLF (3.3K Txn/s). In terms of latency, with the same level of throughput, FISCO-BCOS achieves the lowest latency among the three solutions. When the throughput is below 10K Txn/s, the end-to-end latency of FISCO-BCOS is smaller than half that of HLF and BIDL. In addition, we can observe from Figure 6 that the latency of FISCO-BCOS rises very slowly as the throughput increases quickly.

[3]Note that BIDL demonstrates higher throughput than our experimental results in [46] because of its higher configured testbed with 50 execution nodes.

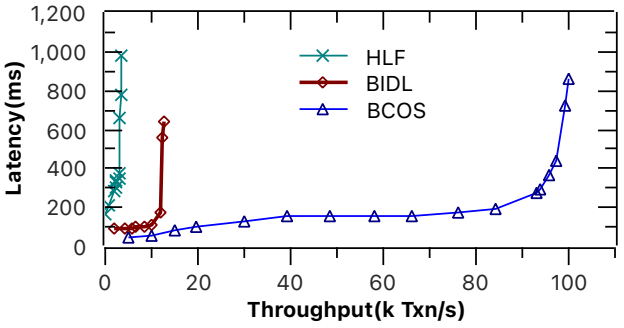
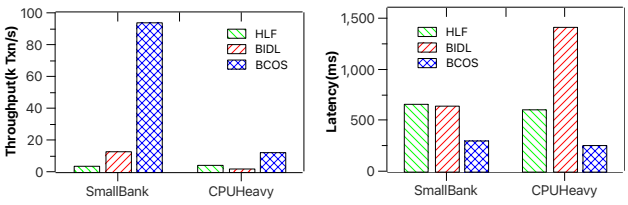



**Figure 6: Performance comparison on SmallBank.**

Secondly, to evaluate the performance in computation-intensive cases, we take experiments with CPUHeavy workload as well, where we compare FISCO-BCOS with HLF and BIDL in quick-sort over an array of 100,000 integers. Again, FISCO-BCOS achieves higher throughput and lower latency than BIDL and HLF as shown in Figure 7. Due to the serial execution of transactions, BIDL has an obvious degradation of performance in the CPUHeavy case compared to the SmallBank workload. In contrast, with the DMC mechanism, FISCO-BCOS remains relatively high throughput and low latency, i.e. 12.1K Txn/s, 200ms respectively. In the case of HLF, its performance increases slightly compared to the SmallBank workload, although it is much lower than FISCO-BCOS in both cases. The increase is due to the fact that the performance of HLF strongly depends on the read/write sets generated by the transactions (since HLF nodes verify the validity of transactions one by one based on the read/write sets during the validation phase), while the CPUHeavy workload actually generates fewer read/write operations than the SmallBank workload.

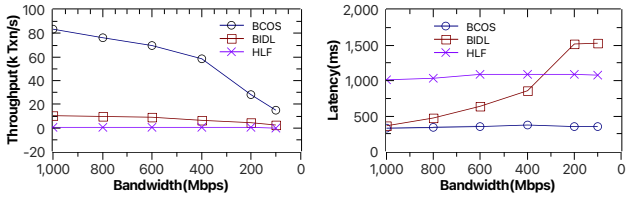
The actual usage of hardware resources depends on the business logic and transaction volume. In this experiment, the machine resources limits for SmallBank and CPUHeavy workloads are Bandwidth and CPU respectively. In the SmallBank experiment, the Bandwidth utilization peaks at more than 800 MBit/s, and CPUHeavy evaluation consumes approximately 30 vCPUs.




(a) Throughput (b) Latency

**Figure 7: Performance comparison on two workloads.**

**Performance in constrained environments.** Different from traditional centralized systems which are deployed entirely by an organization in an environment with a high-speed and high-stability network, such as a datacenter. Permissioned blockchains are usually deployed across multiple organizations over their respective network environments [30]. As an enterprise-grade permissioned blockchain, it is important for FISCO-BCOS to perform well in constrained environments. We conduct FISCO-BCOS performance evaluation on SmallBank over different bandwidths.




(a) Throughput (b) Latency

**Figure 8: Performance over different bandwidths.**

As depicted in Figure 8, we can see that although the performance of FISCO-BCOS decreases as well when the bandwidth becomes more stringent, FISCO-BCOS still has performance gains over BIDL and HLF. With only 100Mbps of bandwidth, FISCO-BCOS still has 15.3K Txn/s throughput and less than 400 ms latency. This is because the impact of the network on the overall performance of the system is greatly reduced due to the pipeline mechanism.

Apart from this, we also deployed 10 nodes in a wide-area network environment for evaluation, with 3, 3, and 4 nodes in each of 3 cities that are more than 1000km away from each other. Again, FISCO-BCOS performs well and obtains 14K Txn/s, 400ms of throughput and latency respectively.

## 7.3 Evaluations of BLP and DMC.

To better understand the advantages of the two key mechanisms in FISCO-BCOS, i.e. the pipeline workflow and the DMC mechanism, we conduct several experiments with different system configurations over SmallBank and CPUHeavy workloads. As shown in Table. 1, both BLP and DMC provide significant improvements on both workloads. However, BLP provides a higher improvement over CPUHeavy on SmallBank, while DMC offers a greater improvement over SmallBank on CPUHeavy. Specifically, BLP improves the throughput of SmallBank and CPUHeavy by 3.6x and 2.2x, respectively. And DMC further improves them by 2.4x and 11x, respectively. We observe that DMC performs very well on CPUHeavy contracts, as it processes intra-block transactions in a highly parallel manner, particularly suitable for CPU-intensive scenarios.

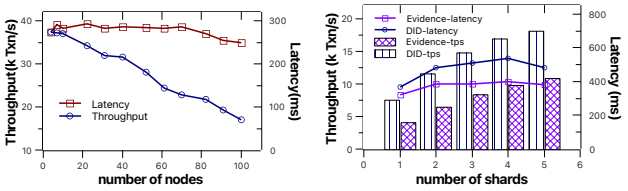
**Table 1: Throughput (k Txn/s) over different optimizations.**

| Workload | Opt-disable | +BLP | +BLP+DMC |
|---|---|---|---|
| SmallBank | 10.8 | 38.8 (3.6x↑) | 93.82 (2.4x↑) |
| CPUHeavy | 0.5 | 1.1 (2.2x↑) | 12.1 (11x↑) |

## 7.4 Scalability Evaluations.

To evaluate the performance of FISCO-BCOS on a large-scale network, we compared the performance of SmallBank workloads with a different number of FISCO-BCOS nodes on our large-scale testbed.

In a real enterprise application scenario, each institutional node involved in building a permissioned blockchain cooperates equally and has a reciprocal status. Therefore, in the large-scale performance evaluation of FISCO-BCOS, we keep each node of the system performing the same duties. All nodes of FISCO-BCOS participate in the entire workflow, which means that each node runs the BFT [13] algorithm to participate in consensus and executes each transaction to get a ledger updated.




(a) Node scalability (b) Shard scalability

**Figure 9: Performance of varying numbers of nodes and shards.**

In this scalability experiment, we increase the number of nodes from 4 to 100 using the SmallBank workload and observe the changes in system throughput while ensuring that FISCO-BCOS performs relatively stable transaction latency (less than 300ms). The results are shown in Figure 9 (a), we can see that the throughput decreases with increasing the number of nodes which is understandable since we use BFT [13] protocol[4] for the ordering phase, however, FISCO-BCOS still achieves satisfactory performance in large scale scenarios, i.e. 17K Txn/s when the node number increases to 100.

In addition to the node scalability experiments, we also performed the scalability evaluation of the shards, i.e. shard scalability. In this experiment, we found that SmallBank and CPUHeavy workloads are not suitable because they are too simple and can be efficiently handled within only one shard. Therefore, here we experimented with two real and much more complex business workloads, namely Evidence and DID [56] (used in Judicial Services, and Cross-border collaboration respectively, see 6). The Evidence application accepts a string of evidence and records it in contracts, while DID performs the issuance of individual certificates. We deploy these two business contracts in 1 to 5 shards and observe their performance to evaluate the scalability of FISCO-BCOS in handling more complex business logic scenarios. As shown in Figure 9 (b), the performance of both applications increases with the increasing number of shards. FISCO-BCOS provides good scalability in real-world applications.

## 8 RELATED WORK

In recent years, researchers from industry and academia have paid much attention to the design and application of permissioned blockchains.

HyperLedger Fabric (HLF) [3] is considered the most commonly used permissioned blockchain system. HLF is a modular framework for developing enterprise-grade applications and industry solutions among mutually untrusted organizations through a consensus protocol such as Raft [42]. Many works [28, 50, 65] are proposed to improve HLF's performance, they mainly focus on reducing the computation and I/O overhead in the ordering and validation phase. HLF utilizes the $Execute \rightarrow Order \rightarrow Validate$ workflow through which transactions are processed concurrently during the execution phase, and guarantee correctness by checking the conflicts in the validation phase and discarding conflicted transactions. However, dropping the conflicted transactions may badly affect the performance. In BCOS, it puts the ordering ahead of the execution, and the parallelism of transaction execution of different nodes is in a deterministic way, avoiding transaction aborts.

Quorum [14] is another widely used system based on Ethereum [22]. It originates from the Ethereum Golang client (geth) [20] and aims to solve the challenges of the financial industry by providing private transactions. Quorum offers much higher throughput than its public chain version. Diem [5] is a decentralized, programmable permissioned blockchain proposed by Facebook. It is designed to support a low-volatility cryptocurrency that will have the ability to serve as an efficient medium of exchange around the world. Dime introduces a novel Move [35] programming language to define the core mechanisms of blockchain, such as the currency and validator membership. Other systems such as Multichain [37], Corda [15], and Tendermint [49] are also carried out in some scenarios. The difference between these systems and BCOS is that these systems all depend on serial processing mechanisms both for block processing and transaction execution. In BCOS, new mechanisms such as block processing workflow pipelining and DMC-based transaction parallel execution are used to enhance the performance of the chain.

Some studies [1, 4, 7, 18, 31, 64] rely on a two-step transaction execution parallelism. A leading node executes transactions first and generates a transaction dependency graph for other validators and therefore the validators can re-execute the transactions based on the generated dependency graph in a parallel manner. In this way, validators have to trust the dependency graph generated by the leader; if the miner is a malicious node, the system cannot reach a consensus. Moreover, validators have to wait and stay idle before receiving the dependency graph from the leader, which reduces the performance of the overall system. BCOS also makes full use of parallelism but does not need a leader.

BIDL [46] proposes a new framework and achieves very high performance in datacenter networks. It leverages the network ordering property in a datacenter network to enable a new shepherded parallel workflow that performs the consensus in parallel with the transaction execution. But unfortunately, some requirements of BIDL, such as triangle property [46] and IP multicasting [60] are easy to be fulfilled in datacenter networks but are difficult to hold true in other networks. BCOS does not have strict limits on the network environment and is insensitive to packet loss and narrow bandwidth as we show in Sec. 7.

Furthermore, some researchers have focused on sharding techniques to improve the scalability and performance of blockchains [2, 16, 32, 39, 58, 63]. However, these efforts are mainly designed for permissionless public blockchains and these approaches are too complex to be applied in permissioned blockchains.

[4]We will try more scalable (i.e., support more nodes) protocols for consensus in the future, such as the VRF and random grouping techniques in Algorand [27]

Last but not least, some classical studies on databases and operating systems also inspire our work, such as SEDA [59], DMT [34, 41], etc.

## 9 CONCLUSION

We propose an enterprise-grade permissioned blockchain system with high performance. By using a block-level pipeline workflow for inter-block processing and a deterministic multi-contract mechanism for intra-block handling, FISCO-BCOS breaks the serial dependency of block processing and realizes parallel execution of transactions, significantly improving overall performance. Our experiments show that FISCO-BCOS outperforms state-of-the-art permissioned blockchains from industry and academia, achieving 7.4 times and 28.4 times the throughput of BIDL and HLF, respectively, with half the latency of them. FISCO-BCOS has already been used in over 300 different real-world industrial scenarios and the source code is available on Github.

## 10 ACKNOWLEDGMENTS

We would like to thank Fuchen Ma, Qiong Luo for their helpful suggestions and all contributors in the FISCO-BCOS community. This work was supported by the National Key R&D Program of China (2021YFB2700300); the National Natural Science Foundation of China (U22B2032, 61972382); the Technology Program of Guangzhou, China (No. 202103050004).

## REFERENCES

[1] Amiri, M. J., Agrawal, D., and El Abbadi, A. Parblockchain: Leveraging transaction parallelism in permissioned blockchain systems. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)* (2019), IEEE, pp. 1337–1347.

[2] Amiri, M. J., Agrawal, D., and El Abbadi, A. Sharper: Sharding permissioned blockchains over network clusters. In *Proceedings of the 2021 International Conference on Management of Data* (2021), pp. 76–88.

[3] Androulaki, E., Barger, A., Bortnikov, V., Cachin, C., Christidis, K., De Caro, A., Enyeart, D., Ferris, C., Laventman, G., Manevich, Y., et al. Hyperledger fabric: a distributed operating system for permissioned blockchains. In *Proceedings of the thirteenth EuroSys conference* (2018), pp. 1–15.

[4] Anjana, P. S., Kumari, S., Peri, S., Rathor, S., and Somani, A. An efficient framework for optimistic concurrent execution of smart contracts. In *2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)* (2019), IEEE, pp. 83–92.

[5] Association, D. Diem website. https://www.diem.com/en-us/, 2023.

[6] Authors, T. Tikv website. https://tikv.org, 2023.

[7] Baheti, S., Anjana, P. S., Peri, S., and Simmhan, Y. Dipetrans: A framework for distributed parallel execution of transactions of blocks in blockchains. *Concurrency and Computation: Practice and Experience* (2019), e6804.

[8] Baliga, A., Subhod, I., Kamat, P., and Chatterjee, S. Performance evaluation of the quorum blockchain platform. *arXiv preprint arXiv:1809.03421* (2018).

[9] Banerjee, A. Everything you need to know about consortium blockchain. https://www.blockchain-council.org/blockchain/everything-you-need-to-know-about-consortium-blockchain/, 2023.

[10] Bartoletti, M., Galletta, L., and Murgia, M. A true concurrent model of smart contracts executions. *international conference on coordination models and languages* (2019).

[11] Bitcoin-core. libsecp256k1. https://github.com/bitcoin-core/secp256k1, 2023.

[12] Castro, M., and Liskov, B. Practical byzantine fault tolerance and proactive recovery. *ACM Transactions on Computer Systems (TOCS) 20* (2002), 398–461.

[13] Castro, M., Liskov, B., et al. Practical byzantine fault tolerance. In *OsDI* (1999), vol. 99, pp. 173–186.

[14] ConsenSys. Goquorum. https://github.com/ConsenSys/quorum, 2022.

[15] Corda. Corda. https://www.corda.net, 2023.

[16] Dang, H., Dinh, T. T. A., Loghin, D., Chang, E.-C., Lin, Q., and Ooi, B. C. Towards scaling blockchain systems via sharding. In *Proceedings of the 2019 international conference on management of data* (2019), pp. 123–140.

[17] Deloitte. How blockchain can reshape trade finance. https://www2.deloitte.com/content/dam/Deloitte/global/Documents/grid/trade-finance-placemat.pdf, 2023.

[18] Dickerson, T., Gazzillo, P., Herlihy, M., and Koskinen, E. Adding concurrency to smart contracts. *Distributed Computing 33*, 3 (2020), 209–225.

[19] Dinh, T. T. A., Wang, J., Chen, G., Liu, R., Ooi, B. C., and Tan, K.-L. Blockbench: A framework for analyzing private blockchains, 2017.

[20] Ethereum. Official golang implementation of the ethereum protocol. https://github.com/ethereum/go-ethereum, 2022.

[21] Ethereum. Ethereum virtual machine (evm). https://ethereum.org/en/developers/docs/evm/, 2023.

[22] Ethereum. Ethereum website. https://ethereum.org/en/, 2023.

[23] Ethereum. Solidity github repository. https://github.com/ethereum/solidity, 2023.

[24] FISCO. Fisco bcos white paper on industrial applications (in chinese). https://www.fisco.com.cn/upload/files/20230713/1689233000761446.pdf, 2022.

[25] FISCO. Fisco bcos website. http://www.fisco-bcos.org, 2023.

[26] Flores-Montoya, A., and Schulte, E. Datalog disassembly. In *29th USENIX Security Symposium (USENIX Security 20)* (2020), pp. 1075–1092.

[27] Gilad, Y., Hemo, R., Micali, S., Vlachos, G., and Zeldovich, N. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th symposium on operating systems principles* (2017), pp. 51–68.

[28] Gorenflo, C., Lee, S., Golab, L., and Keshav, S. Fastfabric: Scaling hyperledger fabric to 20 000 transactions per second. *International Journal of Network Management 30*, 5 (2020), e2099.

[29] Grech, N., Brent, L., Scholz, B., and Smaragdakis, Y. Gigahorse: thorough, declarative decompilation of smart contracts. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)* (2019), IEEE, pp. 1176–1186.

[30] Hyperledger. Case studies:revolutionizing organizations worldwide. https://www.hyperledger.org/learn/case-studies, 2023.

[31] Jin, C., Pang, S., Qi, X., Zhang, Z., and Zhou, A. A high performance concurrency protocol for smart contracts of permissioned blockchain. *IEEE Transactions on Knowledge and Data Engineering* (2021).

[32] Kokoris-Kogias, E., Jovanovic, P., Gasser, L., Gailly, N., Syta, E., and Ford, B. Omniledger: A secure, scale-out, decentralized ledger via sharding. In *2018 IEEE Symposium on Security and Privacy (SP)* (2018), IEEE, pp. 583–598.

[33] Likos, P. How blockchain can transform the financial services industry. https://money.usnews.com/investing/cryptocurrency/articles/how-blockchain-can-transform-the-financial-services-industry, 2021.

[34] Liu, T., Curtsinger, C., and Berger, E. D. Dthreads: efficient deterministic multithreading. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles* (2011), pp. 327–336.

[35] (Meta), F. Move github repository. https://github.com/move-language/move, 2023.

[36] (Meta), F. Rocksdb: A persistent key-value store for fast storage environments. http://rocksdb.org, 2023.

[37] MultiChain. Multichain website. https://www.multichain.com, 2023.

[38] Nakamoto, S., and Bitcoin, A. A peer-to-peer electronic cash system. *Bitcoin.–URL: https://bitcoin. org/bitcoin. pdf 4* (2008).

[39] Nguyen, L. N., Nguyen, T. D., Dinh, T. N., and Thai, M. T. Optchain: optimal transactions placement for scalable blockchain sharding. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)* (2019), IEEE, pp. 525–535.

[40] Nielsen, J. Response times: The 3 important limits. https://www.nngroup.com/articles/response-times-3-important-limits/, 1993.

[41] Olszewski, M., Ansel, J., and Amarasinghe, S. Kendo: efficient deterministic multithreading in software. In *Proceedings of the 14th international conference on Architectural support for programming languages and operating systems* (2009), pp. 97–108.

[42] Ongaro, D., and Ousterhout, J. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (Usenix ATC 14)* (2014), pp. 305–319.

[43] ParityTech. ink! github repository. https://github.com/paritytech/ink, 2023.

[44] Peng, Z., Zhang, Y., Xu, Q., Liu, H., Gao, Y., and Li, X. Neuchain: A fast permissioned blockchain system with deterministic ordering.

[45] Pîrlea, G., Kumar, A., and Sergey, I. Practical smart contract sharding with ownership and commutativity analysis. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (2021), pp. 1327–1341.

[46] Qi, J., Chen, X., Jiang, Y., Jiang, J., Shen, T., Zhao, S., Wang, S., Zhang, G., Chen, L., Au, M. H., et al. Bidl: A high-throughput, low-latency permissioned blockchain framework for datacenter networks. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (2021), pp. 18–34.

[47] Ruan, P., Loghin, D., Ta, Q.-T., Zhang, M., Chen, G., and Ooi, B. C. A transactional perspective on execute-order-validate blockchains. *international conference on management of data* (2020).

[48] Sharma, A., Schuhknecht, F. M., Agrawal, D., and Dittrich, J. Blurring the lines between blockchains and database systems: the case of hyperledger fabric.

*international conference on management of data* (2019).

[49] Tendermint. Tendermint. https://tendermint.com, 2023.

[50] Thakkar, P., and Natarajan, S. Scaling hyperledger fabric using pipelined execution and sparse peers. *arXiv preprint arXiv:2003.05113* (2020).

[51] Tianhua, L., Hongfeng, Z., Guiran, C., and Chuansheng, Z. The design and implementation of zero-copy for linux. In *2008 Eighth International Conference on Intelligent Systems Design and Applications* (2008), vol. 1, IEEE, pp. 121–126.

[52] Tijan, E., Aksentijević, S., Ivanić, K., and Jardas, M. Blockchain technology implementation in logistics. *Sustainability 11*, 4 (2019), 1185.

[53] Treiblmaier, H., and Sillaber, C. The impact of blockchain on e-commerce: A framework for salient research topics. *Electronic Commerce Research and Applications 48* (2021), 101054.

[54] Veronese, G. S., Correia, M., Bessani, A. N., Lung, L. C., and Verissimo, P. Efficient byzantine fault-tolerance. *IEEE Transactions on Computers 62*, 1 (2011), 16–30.

[55] Visa. Visa fact sheet. https://www.visa.co.uk/dam/VCOM/download/corporate/media/visanet-technology/aboutvisafactsheet.pdf, 2023.

[56] W3C. Decentralized identifiers (dids) v1.0. https://www.w3.org/TR/did-core/, 2023.

[57] W3C. Webassembly. https://webassembly.org, 2023.

[58] Wang, J., and Wang, H. Monoxide: Scale out blockchains with asynchronous consensus zones. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)* (2019), pp. 95–112.

[59] Welsh, M., Culler, D., and Brewer, E. Seda: An architecture for well-conditioned, scalable internet services. *ACM SIGOPS operating systems review 35*, 5 (2001), 230–243.

[60] Wikipeida. Ip multicasting. https://en.wikipedia.org/wiki/IP_multicast, 2023.

[61] Xu, C., Zhang, C., Xu, J., and Pei, J. Slimchain: scaling blockchain transactions through off-chain storage and parallel processing. *very large data bases* (2021).

[62] Yin, M., Malkhi, D., Reiter, M. K., Gueta, G. G., and Abraham, I. Hotstuff: Bft consensus with linearity and responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing* (2019), pp. 347–356.

[63] Zamani, M., Movahedi, M., and Raykova, M. Rapidchain: Scaling blockchain via full sharding. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (2018), pp. 931–948.

[64] Zhang, A., and Zhang, K. Enabling concurrency on smart contracts using multiversion ordering. In *Asia-Pacific Web (APWeb) and Web-Age Information Management (WAIM) Joint International Conference on Web and Big Data* (2018), Springer, pp. 425–439.

[65] Zhou, E., Pi, B., Sun, J., Miyamae, T., and Morinaga, M. Performance improvement by using pipelined execution on hyperledger fabric.

# Appendix: Artifact Description/Artifact Evaluation

## ARTIFACT DOI

https://zenodo.org/record/8207532

## ARTIFACT IDENTIFICATION

Title: DXLedger: An Enterprise-grade Permissioned Blockchain System with High-performance

Authors: Huizhong Li, ICT/CAS & WeBank, etc.

FISCO BCOS is a globally known open-source permissioned blockchain platform and has been widely deployed for 300+ industrial and business applications. But due to double-blind rules, we hide the real name of the system and use DXLedger as an alternative name in the full paper submission. The source code repository on GitHub is https://github.com/FISCO-BCOS/FISCO-BCOS.

(i) In this paper, we propose DXLedger, an enterprise-grade permissioned blockchain system with high-performance. Traditional permissioned blockchain systems have two key limitations. 1) At the block level, the serial dependency of inter-block processing severely limits the system throughput. A new block must wait for the completion of all previous blocks. 2) At the transaction level, the lack of efficient intra-block transactions concurrency makes it difficult to achieve high performance, especially when dealing with multiple CPU-heavy contracts which are commonly used in industrial scenarios. DXLedger introduces Block Level Pipelining (BLP) workflow to process blocks in a pipeline manner. In addition, a scheduling algorithm Deterministic Multi-Contract (DMC) is designed to efficiently execute transactions in parallel. Under BLP and DMC, DXLedger achieves inter-block and intra-block paralleling to meet high-performance requirements in industrial application scenarios.

(ii) The artifacts that we made publicly available contain three projects, including DXLedger, HLF, and BIDL, all of which can be used to reproduce all the experiments in this paper. HLF is the most popular enterprise-grade permissioned blockchain platform that is world-widely used in various scenarios. BIDL is a state-of-art work in academia that uses a shepherded parallel workflow to achieve high performance in datacenter networks.

(1) Persistent ID (DOI, GitHub URL, etc.): https://doi.org/10.5281/zenodo.8207532
(2) Artifact name: DXLedger-AD
(3) Citation of artifact (if known): Lihuizhong. (2023). DXLedgerAD (v1.0). Zenodo. https://doi.org/10.5281/zenodo.8207532

(iii) We conduct a series of experiments to evaluate the overall performance of DXLedger and the efficiency of BLP and DMC. We use two popular workloads in BlockBench project, i.e. SmallBank and CPUHeavy smart contracts. Two industrial applications, Evidence and DID, are also included to perform real-world scenarios evaluation. The comparison is performed with two state-of-the-art platforms of industry and academia, Hyperledger Fabric (HLF) and BIDL respectively. All experiments were conducted on two types of cloud servers. Type #1 was equipped with an Intel(R) Xeon(R) Platinum 8378C CPU @ 2.80GHz (16 cores, 32 threads), 64GB of RAM, and Ubuntu 20.04, with a total of 11 servers, all in the same region. (A similar group of servers was deployed to conduct wide-area experiments, consisting of 11 servers, four in Guiyang, three in Beijing, and three in Ulanqab.) Type #2 was equipped with an Intel(R) Xeon(R) Gold 6266C CPU @ 3.00GHz (4 cores and 8 threads), 16GB RAM, and Ubuntu 20.04, with a total of 100 servers. Experiments 1-4 and 6 were run on #1 servers and experiment 5 was run on #2 servers. The experiments reported in this paper are:

(1) We perform an end-to-end performance comparison over the SmallBank workload. In this experiment, we randomly generate a batch of transfer transactions, each involving two accounts belonging to different organizations. The results are shown in Fig 5 which is collected from experiment 1.
(2) To evaluate the performance in computation-intensive cases, we take experiments with CPUHeavy workload as well, where we compare DXLedger with HLF and BIDL in quicksort over an array of 100,000 integers. The results are shown in Fig 6 which is collected from experiment 1 and experiment 2.
(3) Performance in constrained environments: We conduct DXLedger performance evaluation on SmallBank over different bandwidths. The results are shown in Fig 7. Apart from this, we also deployed 10 nodes in a wide-area network environment for evaluation, with 3, 3, and 4 nodes in each of 3 cities that are more than 1000km away from each other. Again, DXLedger performs well and obtains 14K Txn/s, 400ms of throughput and latency respectively which is collected from experiment 3.
(4) Evaluations of BLP and DMC: To better understand the advantages of the two key mechanisms in DXLedger, i.e. the pipeline workflow and the DMC mechanism, we conduct several experiments with different system configurations over SmallBank and CPUHeavy workloads. As shown in Table I which is collected from experiment 4.
(5) Scalability Evaluations: To evaluate the performance of DXLedger on a large scale network, we compared the performance of SmallBank workloads with a different number of DXLedger nodes on our large-scale testbed. In this scalability experiment, we increase the number of nodes from 4 to 100 using the SmallBank workload. The results are shown in Fig 8(a) which is collected from experiment 5.
(6) In addition to the node scalability experiments, we also performed the scalability evaluation of the shards, i.e. shard scalability. In this experiment, we experimented with two real and much more complex business workloads, namely Evidence and DID, The results are shown in Fig 8(b) which is collected from experiment 6.
(7) Experiment 7 tested the performance of BIDL, and Figures 5-7 used the data from Experiment 7
(8) Experiment 8 tested the performance of HLF, and Figures 5-7 used the data from Experiment 8

## REPRODUCIBILITY OF EXPERIMENTS

To facilitate testing, we wrote 'experiments.sh' to run the test cases of DXLedger, and we will describe how each experiment was run one by one below. First of all, we should download the DXLedger-new.zip artifact and decompress it to get 'experiments.sh' and other components.

Because the experiment in the paper requires 10-100 machines to deploy the experimental environment, we provide a stand-alone 4-node environment in DOI for Functional verification, which is located in the 4nodes_functions_check_env, users can execute perf scripts to verify functionality. Due to the competition of hardware resources between the four nodes and the pressure testing tools the results is not If you want to fully replicate the experimental results is not representative, it is recommended to use the hardware resources described in subsequent experiments.

### 0.1 Experiment 1 and 2: End-to-end performance evaluation with SmallBank and CPUHeavy

To ensure 'experiments.sh' work properly, please configure a password-free login for the machine on which the script will be executed. The result of this experiment is used by Fig 5 and Fig 6. This experiment is expected to take 10 minutes and the steps are as follows:

1. Fill server IP where the node is located into 'nodes_ip_array' of 'experiments.sh', and modify 'user' and 'user_home_path' accordingly.

2. Build the blockchain nodes on the 10 prepared machines.

```bash
# Please modify the IP of the 10 prepared machines
# iplist is defined "IP1 IP2 ... IP10"
nodes_ip_array=(iplist)
# generate nodes, nodelist is defined as
# "IP1:1,IP2:1,...,IP10:1"
bash ./build_chain.sh -l iplist -z -e ./fisco-bcos
# deploy nodes
for ip in ${nodes_ip_array[*]}; do
    echo "deploying ${ip}";
    scp nodes/${ip}.tar.gz ${ip}:/data/;
    tar -zxf ${ip}.tar.gz ;
done
```

3. Prepare performance evaluation tools.

```bash
# download sdk performance tools from
# https://github.com/FISCO-BCOS/java-sdk-demo.git
# the tool is already prepared in DOI zip
# build tool use command ./gradlew GoJF clean build -x test
# user only need modify conf/config.toml
# to set right endpoints in [network].peers
cd java-sdk-demo/dist
# config sdk
for ip in ${nodes_ip_array[@]}; do
    if [ -z "${sdk_ip_list}" ];then
        sdk_ip_list="\"${ip}:20200\""
    else
        sdk_ip_list="${sdk_ip_list},\"${ip}:20200\""
    fi
done
# Modify the test IP in the SDK configuration file
cd dist
cp conf/config-example.toml conf/config.toml
sed -i "s/peers=.*/peers=[${sdk_ip_list}]/g"
   \\ conf/config.toml
cp ../../nodes/${nodes_ip_array[0]}/sdk/* conf/
```

4. Perform evaluation.

```bash
# put the experiments.sh in java-sdk-demo/dist
# the -T option specifies the number of transactions
# to be sent in every test, set to 0 means
# totalTransactions=qps*10
bash experiments.sh -T 0
```

5. Collect evaluation results,they are in '${nodes_count}nodes-smallbank-${tx_number}tx-$(date +"%Y%m%d").log'

```bash
cat 10nodes-smallbank-0tx*
```

the result is as follows:

```bash
perf small bank on nodes qps 94000
====== Start , count: 940000, qps:94000,
Load DagTransferUser end, count is 10000
Start userTransfer test...
========================================
Send   : 100% 940000/940000  (0:00:10 / 0:00:00)
Receive: 100% 940000/940000 (0:00:10 / 0:00:00)
========================================
Total transactions:  940000
Total time: 10621ms
TPS(include error requests): 88503.90735335655
TPS(exclude error requests): 88503.90735335655
Avg time cost: 320ms
Errors: 0
Time group:
0    < time <  50ms   : 739  : 0.07861702127659574%
50   < time <  100ms  : 2309  : 0.24563829787234043%
100  < time <  200ms  : 166379  : 17.699893617021274%
200  < time <  400ms  : 545134  : 57.992978723404256%
400  < time <  1000ms : 222994  : 23.72276595744681%
1000 < time <  2000ms : 2445  : 0.2601063829787234%
2000 < time           : 0  : 0.0%
```

The 'TPS(transactions per second)' represents throughput(Txn/s) which is used is Fig 5 and Fig 6 and the 'Avg time cost' represents Latency.

## 0.2 Experiment 3: Performance in constrained environments.

In order for 'experiments.sh' to run properly, configure a password-less login on the machine where the script will be executed. The **experiments steps 1-3 are the same as experiment 1**, you can reuse the environment from experiment 1. The result of this experiment is used by Fig 7. This experiment is expected to take 30 minutes and the steps are as follows:
1. Fill in the 'nodes_ip_array' in 'experiments.sh' with the server IPs where the nodes are located, and modify 'user' and 'user_home_path' accordingly.
2. Create blockchain nodes on the 10 prepared machines.
3. Prepare performance evaluation tools.
4. Execute the test.

```bash
# put the experiments.sh in java-sdk-demo/dist
# the -b option will limit the bandwidth of the network
# to (100 200 400 600 800 1000) Mbit/s to run tests
bash experiments.sh -b
```

5. Collect results,they are in '${nodes_count}nodes-bandwidth-delay-${delay}ms-$(date +"%Y%m%d")'

## 0.3 Experiment 4: Evaluations of BLP and DMC

After closing BLP and DAG corresponding to the following operation, please use the same procedure as in Experiment 1 to perform the performance test, and the experiment is expected to last 30 minutes. The result of this experiment is used by Table I.

(1) Switch off BLP

The default config in this paper of '[consensus].pipeline_size=1000', so we need to modify '[consensus].pipeline_size=1' in config.ini of every node to disable BLP mechanism, details are as follows:

```bash
[consensus]
    min_seal_time=110
    pipeline_size=1
```

(2) Switch off DAG

For each node, set 'enable_dag' as 'false' in 'config.ini'. Then, we can conduct the test by 'experiments.sh -D' as evaluating performance without DMC.

## 0.4 Experiment 5: Nodes Scalability Evaluations

The result of this experiment is used by Fig 8(a). In this experiment, it will automatically deploy 4-100 nodes to the machine for testing, which is expected to take 1 hour, with the following steps:
1. Fill the IP of the machine where the node is located into the 'large_scale_nodes_ip_array' of 'experiments.sh', and modify 'user' and 'user_home_path' accordingly.
2. Prepare the performance test tool.

```bash
cd java-sdk-demo/dist
# the config step is the same as experiment 1.
# The only difference is the number of nodes
```

3. Perform evaluation

```bash
# put the experiments.sh in java-sdk-demo/dist
bash experiments.sh -l
```

4. Collect results,they are in 'large-scale-test-$(date +"%Y%m%d")/'

## 0.5 Experiment 6: Shards Scalability

The result of this experiment is used by Fig 8(b). The **experiments steps 1-3 are the same as experiment 1**, you can reuse the environment from experiment 1. This experiment is expected to take 30 minutes and the steps are as follows:
1. Fill in the 'nodes_ip_array' in 'experiments.sh' with the IPs of the machines where the nodes are located, and modify 'user' and 'user_home_path' accordingly.
2. Create blockchain nodes on the 10 prepared machines.
3. Prepare performance evaluation tools.
4. Execute the test.

```bash
# put the experiments.sh in java-sdk-demo/dist
bash experiments.sh -s
```

## 0.6 Experiment 7: Performance evaluation of BIDL

This experiment is expected to take more than 1 hours (please make sure that the IP-Multicast is supported in your testbed).

1. To configure the machine IPs, edit the bidl/scripts/servers file and change it to the actual machine IPs of the performance test.
2. Set the UDP buffer size so that each machine supports multicast. the BIDL broadcast method uses multicast, so you need to set the UDP buffer size, and the cluster needs to support multicast mode.

```bash
sysctl -w net.core.rmem_max=262144000
sysctl -w net.core.rmem_default=262144000
sudo ifconfig lo multicast
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev lo
```

3. Add the benchmark test of CPUHeavy on top of BIDL, you can pass in the CPUHeavy sorted array length parameter.

```bash
echo "benchmarking..."
cd $normal_node_dir
# default performance test is to evaluate SmallBank workload
if [ $4 == "performance" ]; then
    docker run --name bidl_client --net=host \\
   --cap-add NET_ADMIN normal_node /normal_node/client \\
    --num=100000 --org=$2 --tps=$3
# sort test is for CPUHeavy workload
elif [ $4 == "sort" ]; then
    docker run --name bidl_client --net=host \\
   --cap-add NET_ADMIN normal_node /normal_node/client \\
```

```
    --num=100000 --org=$2 --tps=$3 --sortlength=$6
elif [ $4 == "nd" ]; then
    docker run --name bidl_client --net=host \\
   --cap-add NET_ADMIN normal_node /normal_node/client \\
    --num=100000 --org=$2 --nd=$5 --quiet
elif [ $4 == "contention" ]; then
    docker run --name bidl_client --net=host \\
   --cap-add NET_ADMIN normal_node /normal_node/client \\
    --num=100000 --org=$2 --conflict=$5 --quiet
elif [ $4 == "scalability" ]; then
    docker run --name bidl_client --net=host \\
   --cap-add NET_ADMIN normal_node /normal_node/client \\
     --num=100000 --org=$2 --quiet
else
    echo "Invalid argument."
    exit 1
fi
```

4. Pass in the appropriate parameters and execute the performance test.

```bash
# run with default configuration
# (4 consensus node,50 normal node)
bash run_bidl.sh performance
# To adjust the node type and number of nodes,
# execute the following command.
bash ./bidl/scripts/start_bidl.sh \\
  <1. num of consensus nodes> \\
  <2. num of normal nodes> \\
  <3. peak throughput> \\
  <4. benchmark> \\
  <5. benchmark parameters*>
# Deploy 4 consensus nodes and 10 normal nodes
# for CPUHeavy benchmark test, qps is 100000,
# array length is 100000.
cd bidl/scripts
bash start_bidl.sh 4 10 100 sort 100000
```

5. Collect evaluation results,they are in:/home/$USER/logs/

```bash
# 1. compute TPS
cat normal_0.log | grep "block commit throughput:" |
python3 ../scripts/bidl_tput.py >> tput.log
# 2. compute total timecost
cat normal_0.log | grep "Execution latency" | \\
python3 ../scripts/ bidl_execution_latency.py \\
>> execution_latency.log
# 3. compute commit timecost
cat normal_0.log | grep "Commit latency" | \\
python3 ../scripts/bidl_latency.py \\
>>  commit_latency.log
# 4. compute consensus timecost
cat consensus_0.log | grep "Consensus latency" | \\
python3 ../scripts/consensus_latency.py \\
>> consensus_latency.log
```

## 0.7 Experiment 8: Performance evaluation of HLF

This experiment is expected to take more than 1 hours (deploying an entire runtime environment of HLF is time-consuming).

1. Set IPs of testing machine.

2. Create configurations: Generate config file for HLF, modify 'config-fabric.yaml 'like:

```yaml
project: fabric

logging: INFO
network: HLF
tls: 'false'
crypto: fabric_exp/organizations
artifacts: fabric_exp/channel-artifacts
kafka_dir: fabric_exp/kafka

peers:
  template: peer
  image: hyperledger/fabric-peer:2.3.0
  count: 10
  bootstrap: 0

orderers:
  template: orderer
  image: hyperledger/fabric-orderer:2.3.0
  count: 4

kafkas:
  template: kafka
  image: hyperledger/fabric-kafka
  count: 3

zookeeper:
  name: zookeeper
  image: hyperledger/fabric-zookeeper

cli:
  name: cli
  image: hyperledger/fabric-tools:latest
  chaincode: fabric_exp/chaincode
  scripts: fabric_exp/scripts

tape:
  name: tape
  image: tape:latest
  config: fabric_exp/tape.yaml
  organizations: fabric_exp/organizations

hosts:
   - node01
   - node02
   - node03
   - node04
   ...
```

Execute 'create_artifact.sh ' to generate docker configuration:'docker-compose-fabric.yaml'

```bash
bash create_artifact.sh fabric
```

3. Deploy fabric instances: deploy fabric images to multiple servers using configuration 'docker-compose-fabric.yaml'

```bash
docker stack deploy --resolve-image never \\
    --compose-file=docker-compose-fabric.yaml fabric
```

After that, we can check the services of HLF using 'docker service ls'

4. We use 'tape' for performance testing, modify the tape configuration file: config.yaml

```yaml
# Definition of nodes
peer0: &peer0
  addr: fabric_peer0:7051
peer1: &peer1
  addr: fabric_peer1:7051
peer2: &peer2
  addr: fabric_peer2:7051
peer3: &peer3
  addr: fabric_peer3:7051
orderer1: &orderer1
  addr: fabric_orderer0:7050

endorser_groups: 1
endorsers:
  - *peer0
  - *peer1
  - *peer2
  - *peer3
others:
  - *peer4
  - *peer5
  - *peer6
  - *peer7
committer: *peer0
orderer: *orderer0
# Invocation configs
channel: mychannel
chaincode: smallbank
# args:
num_of_conn: 2
client_per_conn: 1
threads: 10
orderer_client: 200
mspid: Org1MSP
private_key: ./path_to_msp/keystore/priv_sk
sign_cert: ./path_to_msp/signcerts/User1.pem
check_txid: true
check_rwset: false
e2e: false
```

5. Execute the command for performance testing

```bash
tape --e2e -n 50000 --burst 50000 --num_of_conn i \\
   --client_per_conn $j --send_rate $send_rate \\
   --orderer_client $k --groups 5 \\
   --config config.yaml > $log 2>&1
```

And get result 'avg tx cost', 'duration' and 'tps':

```
e2e
transfer money: 10000
read 1000 accounts from ACCOUNTS txnum: 10000
sent num: 0
end num: 10000
txnum: 10000 , avg tx cost: 4024 ms
tx: 10000, duration: 16.0823411045, tps: 621.800019
```

## ARTIFACT DEPENDENCIES REQUIREMENTS

### 0.8 Hardware resources required and utilized

11 virtual machines with following configuration are required for our experiments 0.1-0.3, and 0.5-0.7. For experiment 0.4, we need 100 VMs with 4vCPUs.

- CPU: Intel(R) Xeon(R) Platinum 8378C CPU @ 2.80GHz (16 cores, 32 threads), usage: 2600%-3100%
- Memory: >=8GB, usage: 3.4-4.9 GB
- Disk: SSD, usage: 100GB
- Network: >=1Gbps, usage: 384 MBit/s average, 882 MBit/s peak

### 0.9 Operating systems

Ubuntu 20.04

### 0.10 Software libraries

- Java 8+ is required for executing the experiments.
- Docker is required only for HLF experiments.
- tc is required only for network emulation.
- ssh no password login is required for remote execution.

### 0.11 Input dataset

The input dataset is the workload generated by Java-sdk-demo on https://github.com/FISCO-BCOS/java-sdk-demo.git with tag v3.3.0.

## ARTIFACT INSTALLATION DEPLOYMENT PROCESS

For DxLedger, everything needed for experiments is included in the artifact DOI (https://zenodo.org/record/8207532). Follow the instructions in AD to install and run the artifact.

For HLF and BIDL, we run the experiments as official documents and also offer instructions in 'REPRODUCIBILITY OF EXPERIMENTS' (section 0.6 and 0.7).